

Conditional Loop in Decision Flow

To create a conditional loop in ID where you set a max number of loop iterations but also have the ability to leave the loop earlier, you can use the following concept.

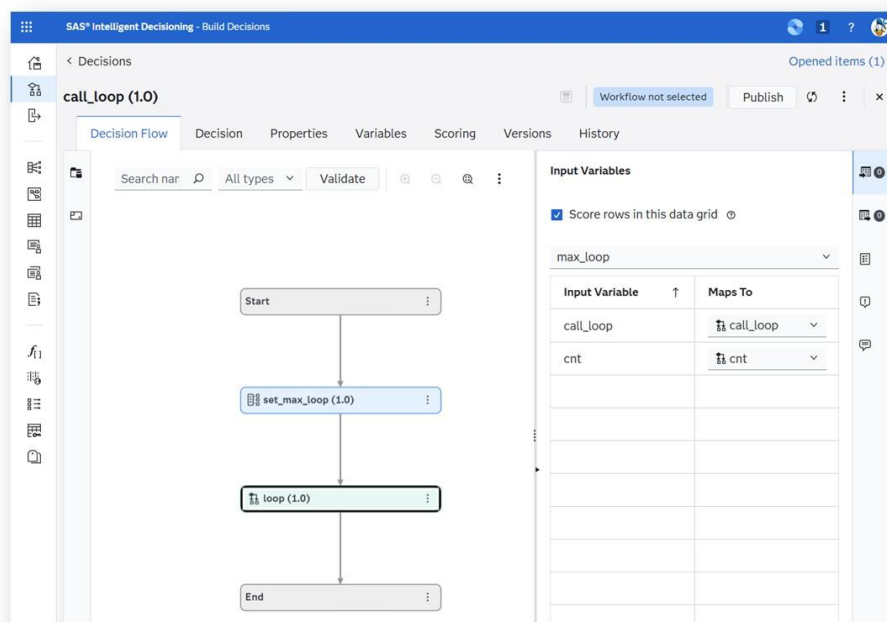
The loop

The loop body is implemented as a sub-decision that is invoked multiple times from a main decision flow. The number of iterations is controlled through a data grid, which acts as the loop controller.

To define the maximum number of iterations, you add one row per potential iteration to the data grid. If the loop needs to terminate before reaching this maximum, an exit flag can be set within the loop body to break out of the loop early.

A rule set is used to create and output the control data grid. During this step, you populate the grid with the required number of rows corresponding to the maximum allowed iterations. The data grid must include at least one “dummy” column, which is only needed to be able to add rows.

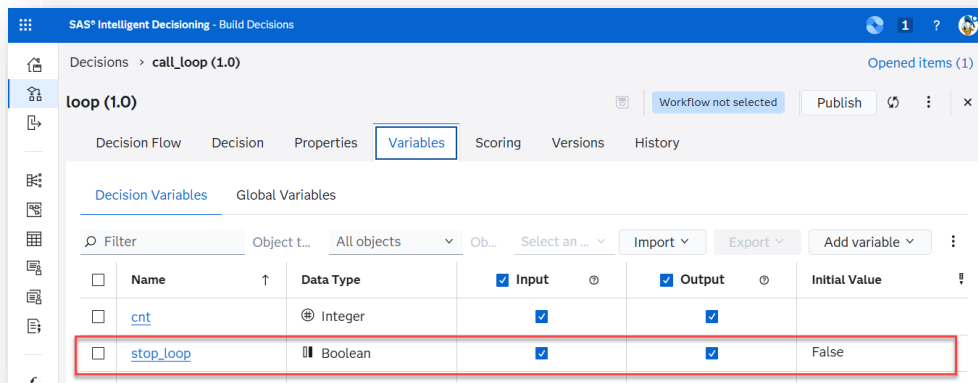
In the main decision flow, you first invoke the rule set that generates the control data grid. This is followed by the loop decision flow and configured using “[Score rows in this data grid](#)”. There is no need to map the “dummy” data grid column to the loop decision flow. This configuration automatically triggers the loop decision flow once for each row in the data grid.



Main Decision

Loop Body

In the loop decision flow, you need one input/output variable (in addition to all other required variables) to control if the loop should stop before the max number of iterations is reached.

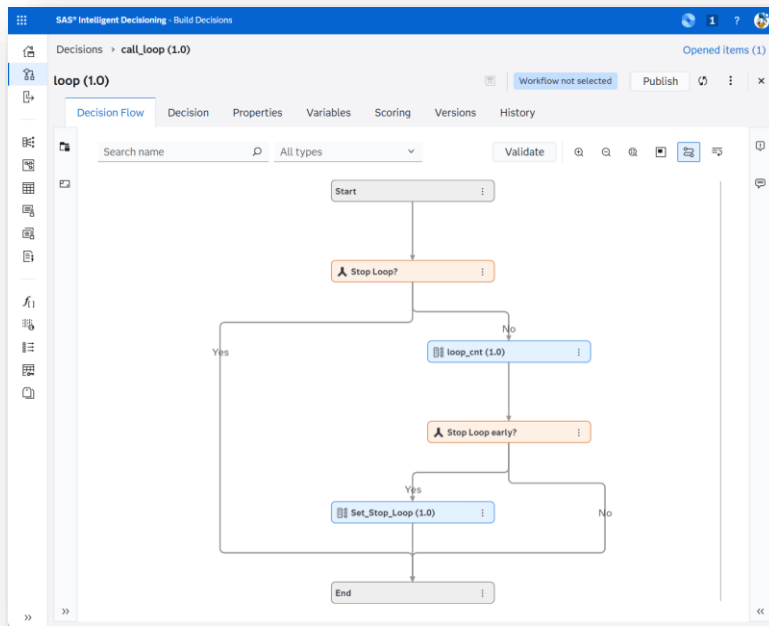


Variable to stop early

At the beginning of the loop body decision flow, a branch is used to evaluate a loop control indicator that determines whether the loop should terminate early. If this indicator signals that no further iterations are required, the flow immediately exits by routing directly to the End-Node.

If the loop is still active, execution continues through the branch that contains the full set of loop-processing nodes. Within this branch, business logic can decide that no additional loop iterations are necessary. When this condition is met, the loop indicator is updated to signal loop termination.

Because the loop indicator is defined as an input/output variable, its updated value is preserved across iterations. As a result, the next time the loop body is invoked, execution is redirected at the top-level branch straight to the End-Node, effectively preventing any further processing. This pattern enables clean, deterministic early exit behaviour while keeping loop control logic centralized and easy to maintain.



Loop Body

Key advantages of this design include:

- A single, clearly defined sub-decision that encapsulates the loop body.
- The maximum number of iterations is externalized rather than hardcoded in the decision logic.
- Adjusting the iteration limit requires only a change to the number of rows in the control data grid.
- The maximum number of iterations can optionally be parameterized to support dynamic configuration.